

Migration **validada de ponta a ponta** num Postgres real: schema cria sem erros, seed insere, e as 3 consultas de negócio retornam inteligência correta. Vou marcar essa etapa e te dar o status — esse é um bom ponto para você revisar o schema e rodá-lo no Supabase.

Update Todos

Criar CLAUDE.md na raiz (consolida skills + comandos)
Fase 2: preencher backend/Dockerfile + .dockerignore
Fase 2: preencher docker-compose.yml (serviço backend)
Fase 2: preencher .github/workflows/ci.yml (ruff + pytest)
Fase 2: fixar versões em requirements.txt e verificar instalação/build/run
Fase 3: migration SQL + seed + consultas — validado em Postgres real
Você: rodar 0001_init.sql e 0002_seed.sql no SQL Editor do Supabase
Fase 3: backend por módulo (auth, instructors, bookings, reports, chat, payments)
Fase 3: testes pytest dos services
Fase 4: app Expo / React Native
Fase 5: revisão contra critérios de aceite

Status — Fases 1, 2 e o banco da Fase 3 prontos e validados



Fase 1 (terreno): CLAUDE.md criado na raiz, consolidando os 6 skills + comandos de build/run/test/git-flow.

Fase 2 (Docker) — validada de ponta a ponta:

- backend/Dockerfile + backend/.dockerignore (base python:3.12-slim)
- docker-compose.yml — serviço backend na rede cnh-net (DB = Supabase nuvem, REQNF01)
- .github/workflows/ci.yml — ruff + pytest em cada push/PR (REQNF04)
- backend/requirements.txt com versões fixadas + backend/pyproject.toml
- ✓ pip install resolveu limpo · ✓ ruff passa · ✓ pytest passa (1 teste) · ✓ docker compose build builda · ✓ contêiner sobe e /health + /docs respondem

Fase 3 — Banco (validado num Postgres real e descartável, sem tocar no seu Supabase):

- backend/db/0001_init.sql — 12 tabelas, 3FN: junção N:N instructor_categories, sem arrays (1FN), sem campos calculados (3FN), constraints UNIQUE/CHECK/FK, e até regra de negócio no banco (chk_vehicle_docs: veículo próprio exige CRLV+foto)
- backend/db/0002_seed.sql — dados de demo (senha de todos: senha123)
- backend/db/0003_consultas_negocio.sql — as 3 consultas da §8.3, testadas e retornando dados

👉 Ação sua agora (em paralelo, enquanto codo o backend)

No SQL Editor do Supabase, rode na ordem: 0001_init.sql e depois 0002_seed.sql. Dá também para abrir o schema no DBeaver para conferir as constraints (a §8.1 sugere isso).

Pontos de defesa para os professores (o schema foi desenhado pensando na arguição rigorosa):

- 1FN → categorias do instrutor não são uma lista numa coluna, viraram a tabela instructor_categories.
- N:N → instrutor↔categoria quebrado em tabela de junção com PK composta.
- 3FN → nota_média e n° de aulas não são colunas; são derivados nas consultas com GROUP BY (por isso as 3 queries existem). bookings não guarda instructor_id (vem do slot).

Quer **revisar/ajustar alguma tabela ou coluna** antes de eu seguir construindo os repositories em cima desse schema? Se estiver bom, eu já avanço com os módulos do backend (auth → instructors → bookings → reports → chat → payments). Posso seguir?

Dúvidas que fiquei:

antes eu usava um ambiente virtual para manter as bibliotecas no meu projeto e não atrapalhar outros. Nunca havia usado antes o docker. Com a entrada dele, como fica? Ainda sempre que vou começar a codar dou um ...\\Scripts\\activate? consulta de negocios

1) venv x Docker — como convivem

Eles resolvem o mesmo problema (isolamento) em dois níveis diferentes:

	venv	Docker
O que isola	só os pacotes Python	o ambiente inteiro : SO + Python + pacotes
Onde roda	direto na sua máquina (host)	dentro de um contêiner separado
Para que serve	seu dia a dia de código	rodar/demonstrar de forma idêntica em qualquer máquina

A regra prática:

- **Codando no dia a dia** → **sim, continue ativando o venv** (`\\.venv\\Scripts\\activate`). É ele que dá autocomplete no editor, faz o `ruff` e o `pytest` rodarem, e te deixa subir a API rápido com `uvicorn app.main:app --reload`. Tudo isso roda **no host**, então precisa do venv.
- **Para testar "como vai rodar de verdade" / demonstrar / garantir o "na minha máquina funcionou"** → **docker compose up**. O contêiner **não usa o seu venv**; ele instala as dependências por conta própria a partir do `requirements.txt` e roda isolado.

Pontos-chave para não confundir:

- Você **nunca ativa venv dentro do Docker** — são mundos separados. O contêiner já nasce "isolado".
- A **fonte da verdade é o requirements.txt**: tanto o venv quanto o Docker instalam dele. Por isso, sempre que adicionar uma biblioteca, ela entra no `requirements.txt`.
- O `venv/` é ignorado pelo git e pelo `.dockerignore` — ele nunca vai para a imagem (senão "contaminaria" o contêiner com coisas do Windows).

Resumo: **venv = ferramenta de desenvolvimento**; **Docker = ferramenta de execução/entrega**. No Ciclo 2 você usa os dois — venv enquanto escreve, Docker para empacotar e apresentar.

2) As 3 consultas de negócio (para o roteiro §8.3)

A "sacada" que liga tudo: como o banco está em **3FN**, a gente **não guarda** valores que dão para calcular (nota média, nº de aulas, médias de competência). Então essas consultas **derivam** essa inteligência na hora — é justamente por isso que elas existem. Esse é o argumento que amarra a §8.3 com a §8.1/8.2.

Q1 — Ranking de instrutores (nota média + nº de aulas).

- **O que faz:** para cada instrutor, calcula a **média das estrelas** que os alunos deram (`AVG(r.stars)`) e **conta** quantas aulas ele já teve confirmadas/concluídas.
- **Como:** parte de `instructor_profiles`, junta com `users` (pegar o nome), desce para `availability_slots` → `bookings` (as aulas daquele instrutor) e `ratings` (as notas). O `LEFT JOIN` garante que um instrutor **sem** aulas ainda apareça (com nota vazia), em vez de sumir.
- **Por que GROUP BY:** para colapsar várias linhas (vários ratings/aulas) em **uma linha por instrutor** com os agregados.
- **Valor de negócio / REQ:** é o que alimenta os **filtros secundários do REQ03** ("ordenar por nota" e "quantidade de aulas concluídas") na busca do aluno.

Q2 — Dashboard de evolução do aluno (médias das 3 competências).

- **O que faz:** para cada aluno, calcula a **média de baliza, percurso e controle de embreagem** ao longo de todas as aulas dele.
- **Como:** parte de `lesson_reports` (os relatórios), junta com `bookings` para saber de **qual aluno** é cada relatório, e com `users` para o nome.
- **Por que GROUP BY:** junta vários relatórios do mesmo aluno em **uma linha** com as três médias.
- **Valor de negócio / REQ:** é literalmente o **Dashboard de Evolução (REQ05/REQ08)** — mostra ao aluno onde ele está bem e onde precisa reforçar, e é a base da "probabilidade de aprovação".

Q3 — Aulas confirmadas por região.

- **O que faz:** conta quantas aulas **confirmadas** existem em cada **bairro/região**.
- **Como:** parte de `bookings`, sobe para `availability_slots` → `instructor_profiles` (de onde vem a `region`), filtrando `status = 'confirmed'`.
- **Por que GROUP BY:** agrupa as aulas por região para ter **um número por bairro**.
- **Valor de negócio:** visão **operacional/admin** — mostra onde está a demanda atendida (útil para o administrador e para a narrativa de mercado do projeto).

Os três resultados reais que rodei no seed: João com **nota 5,0 / 1 aula**; Vicente com médias **baliza 4,0 · percurso 7,0 · embreagem 6,0**; e **Pinheiros = 1 aula confirmada**. Esses números você pode usar como print de exemplo no roteiro.

Segunda ação do claude:

Update Todos

Criar CLAUDE.md na raiz (consolida skills + comandos)
 Fase 2: Docker (Dockerfile, compose, CI, requirements) — validado build/run
 Fase 3: migration SQL + seed + views + consultas
 Fase 3: backend completo (29 endpoints) — ruff + pytest verdes, conexão Supabase OK
 Backend validado end-to-end (read) contra Supabase real via Docker
 Você: rodar 0004_views.sql no Supabase (falta a view instructor_cards p/ a busca)
 Fase 4: app Expo / React Native
 Fase 5: revisão contra critérios de aceite

Backend completo e validado contra o Supabase real

O backend inteiro está pronto, seguindo as camadas dos skills (router → service → repository), com **29 endpoints** cobrindo REQ01–REQ09:

Módulo	Endpoints	REQ
--------	-----------	-----

Auth	register, login (JWT + bcrypt)	REQ01
Users	meu perfil (ver/editar), notificações	REQ01, REQ04
Instructors	solicitar perfil, meu perfil, busca com filtros, perfil público, aprovar	REQ02, REQ03
Bookings	agenda, solicitar aula, aceitar/recusar, política de reembolso	REQ03, REQ04, REQ07, REQ09
Reports & IA	criar relatório, meus relatórios, dashboard, sugestões	REQ08, REQ05
Chat	mensagens, enviar, avaliar 5★	REQ06, REQ07
Payments	comprovante, cancelar+reembolsar	REQ09

Tudo verificado de verdade:

- ✓ `ruff` limpo · ✓ `pytest` **14 testes** passam (política de reembolso, regra dos 8 dias, validações)
- ✓ Conexão real com seu Supabase (via Docker) lendo `categories`, `users`, `instructor_profiles`, `bookings`
- ✓ **End-to-end real**: login do aluno-seed → dashboard correto (baliza 4 / percurso 7 / embreagem 6 / 56,7% / "Baliza")

3 descobertas que resolvi no caminho (todas com correção robusta no código):

1. Cliente Supabase criado no import quebrava testes → tornei **lazy** (proxy).
2. Chave nova `sb_secret_` rejeitada → você trocou pela **service_role** legada (JWT).
3. SSL bloqueado pelo antivírus do Windows → **roda no Docker** (registre no `CLAUDE.md`); e a `SUPABASE_URL` tinha `/rest/v1` sobrando → `config.py` agora **normaliza**.

👉 Falta só uma ação sua para liberar a busca de instrutores

Rode o `backend/db/0004_views.sql` no SQL Editor do Supabase (cria a view `instructor_cards` que a busca do REQ03 usa). Assim que rodar, eu valido a busca end-to-end e seguimos.

Quando você confirmar o `0004`, posso **avançar para a Fase 4 (app Expo/React Native)**. Quer que eu já comece a estruturar o app mobile enquanto isso, ou prefere que eu valide a busca primeiro?

Pq a view `instructor_cards` (ver mlr pq ele chama de view) é a única tabela UNRESTRICTED?

Sobre o "UNRESTRICTED" na view — está correto, e aqui está o porquê

O que significa: "Unrestricted" = o objeto não tem RLS (Row Level Security) protegendo-o de acesso direto pela API pública (chave `anon`).

Por que só a view aparece assim: uma **view não pode ter políticas de RLS próprias** (RLS se aplica a tabelas, não a views) — então o Supabase sempre marca views como "Unrestricted". As suas tabelas base provavelmente aparecem com o indicador de "RLS disabled" separado, por isso a view é a única com esse selo específico.

Por que isso NÃO é um problema na nossa arquitetura (3 motivos):

1. Nosso backend usa a `service_role`, que ignora RLS de qualquer forma. Ligar ou desligar RLS não muda como a nossa API lê os dados.
2. O app mobile nunca fala direto com o Supabase — só com o nosso FastAPI. Ou seja, o caminho da API pública (`anon`) não é usado por nós.
3. A segurança é feita no código do backend (filtragem por ID do usuário), que é uma decisão de arquitetura registrada no skill `20-supabase-access.md`.

Além disso, a `instructor_cards` só expõe dados de "vitrine" do marketplace (nome, região, preço, nota) — nada sensível como CPF ou documentos.

Quando valeria a pena reforçar: se um dia o projeto fosse exposto publicamente com a chave `anon` disponível, alguém poderia ler tabelas via REST direto. Para um projeto **local/acadêmico não publicado**, o risco é baixo.

Opção de hardening (recomendo para a narrativa de LGPD / REQNF02): dá para **ativar RLS nas tabelas base** — principalmente em `users` (que tem CPF). Como o nosso backend usa `service_role` (que bypassa RLS), **ativar RLS não quebra nada** no nosso fluxo, e ainda fortalece o argumento de proteção de dados na hora da apresentação. Se quiser, eu gero um `0005_rls.sql` que liga RLS nas tabelas sensíveis. **Quer que eu faça isso?** (Não é obrigatório agora — posso seguir para a Fase 4 e deixar esse opcional para depois.)

Terminou a fase 4 |(front) e 5 (revisao)

O app está completo e validado até onde dá sem um navegador. Status final:

✅ Fase 4 (frontend Expo) e Fase 5 (revisão) concluídas

App Expo / React Native — 17 telas, 2 personas, tema azul, placeholder de logo:

- **Auth:** Login (com dica dos usuários-seed) + Cadastro (REQ01)
- **Aluno:** Buscar instrutores (filtros + ordenação), Detalhe+Agenda, Pagamento (REQ09+REQ07 endereço), Agendamentos (cancelar/reembolso/política), Dashboard de evolução (REQ05), Perfil, Tornar-se instrutor (REQ02)
- **Instrutor:** Solicitações (aceitar/recusar com motivo — REQ04), Agenda, Relatórios (REQ08), Dashboard dos alunos (REQ05), Perfil
- **Compartilhado:** Chat com polling, localização e avaliação 5★ (REQ06)

Validações que rodei:

Verificação	Resultado
<code>tsc --noEmit</code> (tipos do app)	✅ exit 0
<code>expo export --platform web</code> (bundle)	✅ 584 módulos, build OK
Backend <code>ruff check</code>	✅ All checks passed
Backend <code>pytest</code>	✅ 14 testes
Smoke HTTP aluno (login, /users/me, search, dashboard, bookings)	✅
Smoke HTTP instrutor (/instructors/me, bookings, chat)	✅
<code>docker compose up + /health + /docs</code>	✅

O contrato API↔app foi verificado endpoint a endpoint — tudo que o app chama responde corretamente no Supabase real.

O que falta: você rodar e clicar (não consigo abrir navegador aqui)

```
# Terminal 1 – backend
docker compose up

# Terminal 2 – app
cd frontend
$env:NODE_OPTIONS = "--use-system-ca"
npx expo start --web
```

Login: `vicente@aluno.com` (aluno) ou `joao@instrutor.com` (instrutor), senha `senha123`.
Documentei o workaround do SSL (`--use-system-ca`) e os comandos no novo [frontend/README.md](#), porque seus colegas vão bater no mesmo certificado do antivírus que nos obrigou a usar Docker no backend.

Dois pontos em aberto para você decidir

1. `0005_rls.sql` (opcional): você não respondeu se quer que eu ative RLS nas tabelas sensíveis (reforça a narrativa de LGPD/REQNF02 sem quebrar o backend). Quer?
2. **Teste visual:** assim que você clicar pelos fluxos no navegador, me diga o que achou — ajuste qualquer tela. Aí fechamos a Fase 5 com a sua revisão e o app fica pronto para você preencher o roteiro final.

Quer que eu faça o commit dessa entrega (Fases 2–4) numa branch, ou prefere revisar visualmente primeiro?

Pergunta q fiz:

Para rodar esses comandos no back e front eu preciso abrir um venv?

Resposta

Não, para esses dois comandos **não precisa de venv** em nenhum dos dois. Veja por quê:

Comando	Precisa de venv?	Motivo
<code>docker compose up</code> (backend)	❌ Não	O Docker já é um ambiente isolado próprio — ele instala as dependências dentro do contêiner a partir do <code>requirements.txt</code> . O venv da sua máquina não tem nada a ver com isso. (Esse é, inclusive, o jeito recomendado de rodar, porque resolve o SSL do antivírus.)
<code>npx expo start --web</code> (frontend)	❌ Não	O frontend é Node/JavaScript , não Python. venv é uma ferramenta exclusiva do Python — não se aplica aqui. Só precisa do <code>\$env:NODE_OPTIONS = "--use-system-ca"</code> .

Quando o venv é necessário, então? Só num caso: se você quiser rodar o backend **direto na sua máquina** (sem Docker), com:

```
cd backend
.\venv\Scripts\activate
uvicorn app.main:app --reload      # ou pytest, ou ruff check .
```

Aí sim — porque esses comandos rodam Python no host e precisam das bibliotecas isoladas no venv. Mas como na sua máquina o backend via venv falha ao conectar no Supabase (o tal `CERTIFICATE_VERIFY_FAILED` do antivírus), **prefira sempre o Docker** para rodar o backend.

Resumindo a sua rotina de demonstração:

```
# Terminal 1 — backend (sem venv)
docker compose up

# Terminal 2 — frontend (sem venv)
cd frontend
$env:NODE_OPTIONS = "--use-system-ca"
npx expo start --web
```

O venv fica reservado só para quando você for **escrever/testar código Python** localmente (rodar `pytest/ruff` no dia a dia). Para apenas rodar e apresentar o app, esqueça o venv. 🍌

Entrei no link q aparece qnd rodo o comando do backend. Mas n dá nda:

Quase — duas correções:

1. **Não use `0.0.0.0`**. Esse endereço significa "escutar em todas as interfaces" **dentro do contêiner** — não é um link de navegador. Do seu lado, use sempre **localhost**.
2. **A raiz / não tem página**. O backend é uma **API**, não um site — não existe "home". Se você abrir `http://localhost:8000/` vai ver um `{"detail": "Not Found"}` (404), e isso é **normal**.

Os links úteis do backend são:

URL	O que é
http://localhost:8000/docs	 Swagger — a documentação interativa da API (é isso que você quer abrir para "ver" o backend e testar endpoints). Ótimo para mostrar aos professores (§6.2 do roteiro).
http://localhost:8000/api/v1/health	Health check → <code>{"status": "ok"}</code>

Importante: o backend é só o "motor". A **interface que você vai clicar** (telas, login, busca) é o **app Expo**, que roda separado. Deixe esse `docker compose up` rodando nesse terminal e, em **outro terminal**:

```
cd frontend
$env:NODE_OPTIONS = "--use-system-ca"
npx expo start --web
```

Esse comando do Expo aí sim vai abrir (ou te dar um link tipo `http://localhost:8081`) com a **cara do aplicativo**. É lá que você loga com `vicente@aluno.com / senha123`.

Resumo: `localhost:8000/docs` = ver/testar a API · `localhost:8081` (Expo) = usar o app.